

# Лабораторная работа №5

## Разработка веб-приложения на Spring Boot. Основы Spring Security

### Цель работы

Изучить основы обеспечения безопасности в Spring Boot-приложении: научиться настраивать аутентификацию и авторизацию пользователей, ограничивать доступ к ресурсам приложения, работать с ролями и механизмами защиты запросов, а также познакомиться с основными подходами к хранению состояния пользователя в защищенной системе.

### Теоретическая часть

Spring Security — это основной механизм защиты приложений в экосистеме Spring. Он отвечает за две ключевые задачи:

- **Аутентификация** — проверка личности пользователя. Система отвечает на вопрос: кто выполняет запрос?
- **Авторизация** — проверка прав доступа. Система отвечает на вопрос: что именно пользователю разрешено делать?

После подключения Spring Security приложение начинает пропускать входящий запрос через цепочку фильтров. Эти фильтры проверяют, требуется ли аутентификация, есть ли у пользователя права на доступ и каким способом нужно обработать запрос: через сессию, форму логина, Basic Auth, JWT и т. д.

В этой лабораторной работе мы будем развивать приложение системы уведомлений, созданное в лабораторной №4. Там приложение уже научилось работать с PostgreSQL, сущностями `User` и `Notification`, репозиториями Spring Data JPA, транзакциями и валидацией. Теперь мы добавим в него безопасность: регистрацию пользователя, хранение пароля в зашифрованном виде, роли пользователей, закрытые и открытые URL, а затем рассмотрим разницу между сессионной аутентификацией и JWT.

### Основные элементы Spring Security

- `SecurityFilterChain` — описывает правила безопасности и набор фильтров, которые применяются к запросам.
- `AuthenticationManager` — центральный компонент, запускающий процесс аутентификации.
- `AuthenticationProvider` — выполняет конкретную проверку учетных данных пользователя.
- `UserDetailsService` — загружает пользователя из базы данных.
- `UserDetails` — объект, который Spring Security использует как представление текущего пользователя.
- `PasswordEncoder` — шифрует пароль при регистрации и проверяет его при входе.
- `SecurityContext` — хранит информацию об аутентифицированном пользователе во время выполнения запроса.

- `Filter` — промежуточный обработчик запроса. Фильтры в Spring Security отвечают за чтение токенов, логин, сохранение контекста и другие этапы безопасности.

## Что произойдет после подключения Spring Security

После добавления зависимости `spring-boot-starter-security` Spring Boot автоматически включает базовую защиту приложения:

- все URL становятся закрытыми по умолчанию;
- Spring генерирует стандартную страницу логина;
- создается временный пользователь `user` и случайный пароль, который выводится в логах;
- запросы начинают проходить через цепочку security-фильтров.

На следующем этапе разработчик должен настроить систему под свою задачу: выбрать способ логина, определить открытые и закрытые URL, подключить пользователей из базы данных и назначить им роли.

## Ход выполнения работы

### Часть 0. Подготовка

1. Используйте проект `spring-lab3-notifications`, расширенный в лабораторной №4.
2. Убедитесь, что приложение успешно работает с PostgreSQL.
3. Проверьте, что таблицы `users` и `notifications` уже создаются и данные можно добавлять через Postman.
4. Создайте отдельную ветку проекта или резервную копию перед внесением изменений.

### Часть 1. Подключение Spring Security

1. Добавьте в `pom.xml` зависимость Spring Security:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

1. Для части с JWT добавьте зависимости:

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.6</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
```

Роль зависимостей:

- `spring-boot-starter-security` — подключает базовый механизм Spring Security.
  - `jjwt-api` — API для создания и чтения JWT.
  - `jjwt-impl` — внутренняя реализация работы с JWT.
  - `jjwt-jackson` — сериализация и десериализация JWT payload в JSON.
- Запустите приложение и обратите внимание: теперь все URL закрыты, а в логах появится временный пароль.

## Часть 2. Расширение модели пользователя

В лабораторной №4 сущность `User` хранила имя, email, телефон и другие данные, но не содержала информации для аутентификации. Теперь расширим модель, добавив пароль и роль.

### 2.1. Создайте перечисление роли пользователя

```
package org.example.model.enums;

public enum UserRole {
    ROLE_USER,
    ROLE_ADMIN
}
```

### 2.2. Обновите сущность `User`

```
package org.example.model.entity;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import org.example.model.enums.UserRole;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Getter
@Setter
@NoArgsConstructor
```

```

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String password;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private UserRole role;

    private String phone;

    private String deviceToken;

    private String telegramChatId;

    private LocalDateTime createdAt;

    @OneToMany(mappedBy = "recipient", cascade = CascadeType.ALL)
    private List<Notification> notifications = new ArrayList<>();
}

```

Теперь `email` будет использоваться как логин, `password` — как пароль, а `role` определяет права пользователя.

### 2.3. Обновите `UserDto`

На этом этапе лучше разделить DTO для разных задач. Для обычной работы с пользователем и для регистрации удобнее использовать отдельные классы.

Создайте `RegisterRequest`:

```

package org.example.model.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import lombok.Getter;
import lombok.Setter;

```

```

@Getter
@Setter
public class RegisterRequest {

    @NotBlank(message = "Имя не должно быть пустым")
    private String name;

    @NotBlank(message = "Email обязателен")
    @Email(message = "Некорректный формат email")
    private String email;

    @NotBlank(message = "Пароль обязателен")
    @Size(min = 6, message = "Пароль должен содержать не менее 6 символов")
    private String password;
}

```

Создайте DTO для логина:

```

package org.example.model.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class LoginRequest {

    @NotBlank(message = "Email обязателен")
    @Email(message = "Некорректный формат email")
    private String email;

    @NotBlank(message = "Пароль обязателен")
    private String password;
}

```

### Часть 3. Подготовка репозитория и загрузки пользователя

Spring Security должен уметь находить пользователя в базе данных и преобразовывать его в объект, с которым система безопасности умеет работать. По аналогии с подходом, используемым в отдельном приложении с `UserDetailsService`, удобно вынести эту логику в два класса:

- `CustomUserDetails` — адаптер между нашей сущностью `User` и интерфейсом `UserDetails`;
- `CustomUserDetailsService` — сервис, который загружает пользователя из базы данных и возвращает объект `CustomUserDetails`.

Такой подход делает код более понятным: сущность `User` отвечает только за данные приложения, а детали интеграции со Spring Security сосредоточены в отдельных классах. Подобная организация часто используется в учебных и практических проектах по Spring Security. ([github.com](https://github.com))

### 3.1. Обновите `UserRepository`

```
package org.example.repository;

import org.example.model.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
}
```

### 3.2. Создайте класс `CustomUserDetails`

```
package org.example.security;

import lombok.AllArgsConstructor;
import org.example.model.entity.User;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import java.util.Collection;
import java.util.List;

@AllArgsConstructor
public class CustomUserDetails implements UserDetails {

    private final User user;

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(new SimpleGrantedAuthority(user.getRole().name()));
    }

    @Override
    public String getPassword() {
        return user.getPassword();
    }

    @Override
```

```

    public String getUsername() {
        return user.getEmail();
    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return true;
    }

    public User getUser() {
        return user;
    }
}

```

Что делает этот класс:

- хранит ссылку на нашу сущность `User`;
- возвращает email как логин пользователя;
- возвращает хэш пароля из базы данных;
- преобразует роль пользователя в `GrantedAuthority`, понятный Spring Security;
- позволяет при необходимости получить исходную сущность через `getUser()`.

Именно такой класс играет роль адаптера между моделью приложения и системой безопасности. Spring Security ожидает объект `UserDetails`, поэтому мы отдельно описываем, как из нашей сущности получить логин, пароль и роли. Интерфейс `UserDetails` как раз предназначен для представления пользователя в механизме аутентификации и авторизации. ([docs.spring.io](https://docs.spring.io))

### 3.3. Создайте класс `CustomUserDetailsService`

```

package org.example.security;

import lombok.RequiredArgsConstructor;
import org.example.model.entity.User;
import org.example.repository.UserRepository;
import org.springframework.security.core.userdetails.UserDetails;

```

```

import org.springframework.security.core.userdetails.UserDetailsService;
import
org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
        User user = userRepository.findByEmail(username)
            .orElseThrow(() -> new
UsernameNotFoundException("Пользователь не найден"));

        return new CustomUserDetails(user);
    }
}

```

Что здесь происходит:

- Spring Security передает в метод `loadUserByUsername()` логин пользователя;
- мы ищем пользователя в базе данных по `email`;
- если пользователь найден, создаем объект `CustomUserDetails`;
- если пользователь не найден, выбрасываем `UsernameNotFoundException`.

Важно: `UserDetailsService` не проверяет пароль самостоятельно. Его задача — только загрузить пользователя по логину. Сравнение пароля выполняет `DaoAuthenticationProvider` через `PasswordEncoder`. Именно поэтому `UserDetailsService` считается read-only DAO-слоем для механизма аутентификации. ([docs.spring.io](https://docs.spring.io))

### 3.4. Почему это удобнее, чем возвращать встроенный `User`

Можно было бы вернуть объект `org.springframework.security.core.userdetails.User` прямо из `CustomUserDetailsService`, но отдельный класс `CustomUserDetails` дает несколько преимуществ:

- проще расширять логику в будущем;
- можно сохранить доступ к исходной сущности `User`;
- код становится ближе к реальным проектам, где часто нужны дополнительные поля пользователя;
- лучше видно границу между сущностью базы данных и адаптером безопасности.

## Часть 4. Шифрование паролей и конфигурация безопасности

Пароли нельзя хранить в открытом виде. При регистрации пароль должен шифроваться, а при логине — сравниваться через тот же механизм шифрования.



#### 4.1. Создайте конфигурационный класс SecurityConfig

```
package org.example.config;

import lombok.RequiredArgsConstructor;
import org.example.security.CustomUserDetailsService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final CustomUserDetailsService customUserDetailsService;

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public DaoAuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(customUserDetailsService);
        provider.setPasswordEncoder(passwordEncoder());
        return provider;
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration configuration) throws Exception {
        return configuration.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
```

```

Exception {
    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/auth/**").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .requestMatchers("/users/**").hasAnyRole("USER",
"ADMIN")
            .requestMatchers("/notifications/
**").hasAnyRole("USER", "ADMIN")
            .anyRequest().authenticated()
        )
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
        )
        .authenticationProvider(authenticationProvider())
        .formLogin(form -> form.disable())
        .httpBasic(httpBasic -> {});

    return http.build();
}
}

```

#### 4.2. Что делает эта конфигурация

- `PasswordEncoder` — создает объект для шифрования паролей через BCrypt.
- `DaoAuthenticationProvider` — говорит Spring Security, что пользователя нужно загружать из базы данных через `CustomUserDetailsService` и сравнивать пароль через `PasswordEncoder`.
- `AuthenticationManager` — используется при ручной аутентификации, например в `AuthController`.
- `SecurityFilterChain` — задает правила безопасности:
- `/auth/**` доступны всем;
- `/admin/**` доступны только администраторам;
- `/users/**` и `/notifications/**` доступны аутентифицированным пользователям с ролями `USER` и `ADMIN`;
- остальные URL требуют аутентификации.
- `csrf.disable()` — для REST API на этапе лабораторной отключаем CSRF, чтобы не усложнять отправку запросов через Postman.
- `httpBasic()` — разрешает простой способ логина через заголовок `Authorization: Basic ...`.

### Часть 5. Регистрация пользователя

Чтобы пользователь мог войти в систему, его нужно сначала зарегистрировать и сохранить пароль в зашифрованном виде.

### 5.1. Создайте AuthService

```
package org.example.service;

import lombok.RequiredArgsConstructor;
import org.example.model.dto.RegisterRequest;
import org.example.model.entity.User;
import org.example.model.enums.UserRole;
import org.example.repository.UserRepository;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;

@Service
@RequiredArgsConstructor
public class AuthService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public void register(RegisterRequest request) {
        User user = new User();
        user.setName(request.getName());
        user.setEmail(request.getEmail());
        user.setPassword(passwordEncoder.encode(request.getPassword()));
        user.setRole(UserRole.ROLE_USER);
        user.setCreatedAt(LocalDateTime.now());
        userRepository.save(user);
    }
}
```

### 5.2. Создайте AuthController

```
package org.example.controller;

import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.example.model.dto.RegisterRequest;
import org.example.service.AuthService;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/auth")
@RequiredArgsConstructor
public class AuthController {
```

```

private final AuthService authService;

@PostMapping("/register")
public String register(@RequestBody @Valid RegisterRequest request) {
    authService.register(request);
    return "Пользователь успешно зарегистрирован";
}
}

```

### 5.3. Проверьте регистрацию

В Postman выполните запрос:

- POST `http://localhost:8080/auth/register`

Пример JSON:

```

{
  "name": "Иван Иванов",
  "email": "ivan@example.com",
  "password": "qwerty123"
}

```

Проверьте в базе данных, что:

- пользователь появился в таблице `users`;
- пароль сохранен в виде хэша, а не как открытый текст;
- роль пользователя установлена в `ROLE_USER`.

## Часть 6. Базовая аутентификация через HTTP Basic

На этом этапе у нас уже есть зарегистрированный пользователь. Самый простой способ проверить защиту — использовать HTTP Basic.

### 6.1. Как это работает

- клиент отправляет заголовок `Authorization: Basic ...`;
- Spring Security извлекает логин и пароль;
- `AuthenticationManager` запускает проверку;
- `DaoAuthenticationProvider` обращается к `CustomUserDetailsService`;
- `PasswordEncoder` сравнивает введенный пароль с хэшем в базе;
- если проверка успешна, пользователь считается аутентифицированным.

### 6.2. Проверьте закрытые URL

Попробуйте выполнить без авторизации:

- GET `http://localhost:8080/users/all`
- GET `http://localhost:8080/notifications/all`

Вы получите ошибку доступа.

Теперь в Postman на вкладке **Authorization** выберите тип **Basic Auth**, укажите:

- Username: `ivan@example.com`
- Password: `qwerty123`

После этого повторите запросы. Теперь доступ должен быть разрешен.

## Часть 7. Авторизация по ролям

Добавим различие между обычным пользователем и администратором.

### 7.1. Ограничение URL по ролям

В конфигурации это уже сделано:

```
.requestMatchers("/admin/**").hasRole("ADMIN")  
.requestMatchers("/users/**").hasAnyRole("USER", "ADMIN")
```

Важно: в коде роль хранится как `ROLE_ADMIN`, но в методах `hasRole("ADMIN")` префикс `ROLE_` добавляется автоматически.

### 7.2. Пример контроллера администратора

```
package org.example.controller;  
  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RequestMapping;  
import org.springframework.web.bind.annotation.RestController;  
  
@RestController  
@RequestMapping("/admin")  
public class AdminController {  
  
    @GetMapping("/ping")  
    public String ping() {  
        return "Только для администратора";  
    }  
}
```

Если пользователь с ролью `ROLE_USER` попытается обратиться к `/admin/ping`, он получит ошибку 403.

### 7.3. Методная авторизация

Дополнительно можно защищать не только URL, но и методы сервиса:

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) {
    ...
}
```

Для этого в конфигурации и была добавлена аннотация `@EnableMethodSecurity`.

## Часть 8. Как работают фильтры Spring Security

Spring Security — это не один класс, а цепочка фильтров.

Примерно так проходит запрос:

1. Клиент отправляет HTTP-запрос.
2. Запрос попадает в servlet-container.
3. Контейнер передает его в `DelegatingFilterProxy`.
4. `DelegatingFilterProxy` направляет запрос в `FilterChainProxy`.
5. Далее последовательно срабатывают security-фильтры.
6. Если аутентификация и авторизация успешны, запрос доходит до контроллера.

Наиболее важные фильтры для базового понимания:

- `UsernamePasswordAuthenticationFilter` — обрабатывает логин через форму.
- `BasicAuthenticationFilter` — обрабатывает HTTP Basic.
- `SecurityContextPersistenceFilter` — сохраняет или загружает данные о пользователе из сессии.
- `ExceptionHandlerFilter` — переводит ошибки безопасности в HTTP-ответы.
- `FilterSecurityInterceptor` — выполняет финальную проверку доступа к URL.

На практике это означает: до контроллера запрос доходит только после того, как будет проверено, кто пользователь и какие у него права.

## Часть 9. Сессии в Spring Security

Сессионная аутентификация — это классический stateful-подход.

### 9.1. Как работает сессия

1. Пользователь успешно проходит аутентификацию.
2. Spring Security сохраняет объект `Authentication` в `SecurityContext`.
3. Этот контекст помещается в HTTP-сессию.
4. Клиент получает cookie, например `JSESSIONID`.
5. В следующих запросах браузер автоматически отправляет cookie, и сервер восстанавливает пользователя из сессии.

### 9.2. Особенности сессий

Плюсы:

- удобно для классических web-приложений;

- сервер сам хранит информацию о пользователе;
- нет необходимости пересчитывать токен на каждом запросе.

Минусы:

- сервер хранит состояние;
- сложнее масштабировать приложение без общей сессии;
- хуже подходит для REST API и мобильных клиентов.

### 9.3. Как это связано с нашим приложением

Если бы мы включили `formLogin()`, Spring Security создал бы стандартную страницу логина и после успешного входа сохранял бы пользователя в сессии через `JSESSIONID`. В лабораторной мы используем `httpBasic()` как простой старт, но логика `SecurityContext` и session-based хранения состояния важна для понимания архитектуры.

## Часть 10. JWT: stateless-подход

В REST API часто используют JWT вместо сессии. В этом случае сервер не хранит состояние пользователя между запросами.

### 10.1. Идея JWT

1. Пользователь отправляет логин и пароль на `/auth/login`.
2. Сервер проверяет их и создает токен JWT.
3. Клиент сохраняет токен.
4. При каждом следующем запросе клиент отправляет заголовок:

```
Authorization: Bearer <jwt>
```

1. Сервер проверяет токен и заново создает объект `Authentication` для текущего запроса.

### 10.2. Пример `JwtService`

```
package org.example.security;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;

@Service
public class JwtService {

    private final String secret =
        "verySecretKeyForJwtTokenVerySecretKey12345";
```

```

    public String generateToken(String username) {
        SecretKey key =
            Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));

        return Jwts.builder()
            .subject(username)
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + 1000 * 60
* 60))
            .signWith(key, SignatureAlgorithm.HS256)
            .compact();
    }
}

```

### 10.3. Логин с выдачей JWT

```

package org.example.controller;

import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.example.model.dto.LoginRequest;
import org.example.security.JwtService;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/auth")
@RequiredArgsConstructor
public class JwtAuthController {

    private final AuthenticationManager authenticationManager;
    private final JwtService jwtService;

    @PostMapping("/login")
    public String login(@RequestBody @Valid LoginRequest request) {
        authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                request.getEmail(),
                request.getPassword()
            )
        );

        return jwtService.generateToken(request.getEmail());
    }
}

```

Теперь после логина пользователь получает токен.



## 10.4. JWT-фильтр

Чтобы сервер умел читать JWT из заголовка, нужен фильтр:

```
package org.example.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws
ServletException, IOException {

        String authHeader = request.getHeader("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        String token = authHeader.substring(7);
        String username = jwtService.extractUsername(token);

        if (username != null &&
SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails userDetails =
userDetailsService.loadUserByUsername(username);
            UsernamePasswordAuthenticationToken authenticationToken =
                new UsernamePasswordAuthenticationToken(userDetails,
```

```

null, userDetails.getAuthorities());

SecurityContextHolder.getContext().setAuthentication(authenticationToken);
    }

    filterChain.doFilter(request, response);
}
}

```

Для полноценной работы здесь понадобится добавить в `JwtService` метод `extractUsername()` и проверку токена. На уровне лабораторной достаточно понять саму идею: JWT-фильтр извлекает токен, читает пользователя, формирует `Authentication` и помещает его в `SecurityContext`.

## 10.5. Конфигурация для JWT

При переходе на JWT приложение обычно становится stateless:

```

.sessionManagement(session -> session
    .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
)

```

И JWT-фильтр добавляется в цепочку фильтров до стандартного фильтра логина:

```

http.addFilterBefore(jwtAuthenticationFilter,
    UsernamePasswordAuthenticationFilter.class);

```

## Часть 11. Сравнение сессий и JWT

### Сессии

- сервер хранит состояние;
- используется cookie `JSESSIONID`;
- хорошо подходят для браузерных приложений с формой логина.

### JWT

- сервер не хранит состояние;
- клиент сам носит токен в каждом запросе;
- хорошо подходит для REST API, мобильных приложений и микросервисов.

## Часть 12. Проверка работы приложения

### 12.1. Проверка регистрации

- `POST http://localhost:8080/auth/register`

## 12.2. Проверка Basic Auth

- GET `http://localhost:8080/users/all`
- GET `http://localhost:8080/notifications/all`

Сначала без авторизации, затем с Basic Auth.

## 12.3. Проверка ролей

- GET `http://localhost:8080/admin/ping`

Проверьте доступ под обычным пользователем и под администратором.

## 12.4. Проверка JWT

- POST `http://localhost:8080/auth/login`
- скопируйте полученный токен;
- добавьте в заголовки:

```
Authorization: Bearer <ваш_токен>
```

- повторите запросы к защищенным URL.

## Самостоятельные задания

1. Добавьте отдельный endpoint регистрации администратора.
2. Реализуйте проверку уникальности email при регистрации.
3. Добавьте обработку ошибки при неверном логине и пароле.
4. Вынесите преобразование сущности `User` в DTO в отдельный mapper.
5. Реализуйте метод `extractUsername()` в `JwtService`.
6. Добавьте проверку срока действия токена.
7. Настройте `SessionCreationPolicy.STATELESS` и полностью переключите приложение на JWT.
8. Защитите метод удаления пользователя через `@PreAuthorize("hasRole('ADMIN')")`.

## Контрольные вопросы

1. Что такое аутентификация и чем она отличается от авторизации?
2. Что делает `SecurityFilterChain`?
3. Зачем нужен `UserDetailsService`?
4. Для чего используется `PasswordEncoder`?
5. Почему пароли нельзя хранить в открытом виде?
6. Какую роль выполняет `AuthenticationManager`?
7. Что делает `DaoAuthenticationProvider`?
8. Как работает HTTP Basic?
9. Что хранится в `SecurityContext`?
10. Как работает аутентификация через сессию?
11. Что такое JWT и чем он отличается от сессии?
12. Зачем нужен JWT-фильтр?
13. Что делает `addFilterBefore()` в конфигурации?

14. В чем разница между `SessionCreationPolicy.IF_REQUIRED` и `SessionCreationPolicy.STATELESS` ?
15. Как проверить доступ к URL по роли пользователя?